Adding Swarm node support to your firmware

Oct 14, 2026 @Nightshade

What you're building

To join a Swarm, your board becomes a Bluetooth LE peripheral that speaks a short text-line protocol to the Swarm Android app. Your firmware keeps doing its own Wi-Fi and BLE scanning; Swarm adds a link that tells it when to scan, what channels to cover, and takes its sightings with GPS from the phone.

Every node talks straight to the phone (a star, no chain). The phone numbers boards in the order they first join (node 1, node 2, …), sends each its share of the channels, starts and stops scans, and stamps every sighting with the phone's GPS into one WiGLE CSV.

What your board must do, in short:

- Advertise the Swarm service and expose three GATT characteristics.
- Answer the hello, then send a heartbeat line every 3 seconds.
- Start and stop scanning when told, and send each sighting as one line with a sequence number.
- Keep sightings until the phone acknowledges them, so nothing is lost when the phone is out of range.

Optional but recommended: follow the channel plan the phone sends, and the owner lock (so your board only works for its owner's phone).

The app only takes boards that finish the hello and send heartbeats. A board that skips the hello is ignored. This guide covers the connected (GATT) link, which is all a node needs; it doesn't cover Swarm's own connectionless broadcast link or its firmware-update channel.

Bluetooth setup

Advertise the service UUID with a connectable advert; the phone finds boards by the UUID. Name the board `WDU` before it has a node number and `WDN<number>` after (for example `WDN3`). The name is only for people, but keep that pattern.

Characteristic	UUID	Properties	Direction
Service	`4fafc201-1fb5-459e-8fcc-c5c9c331914b`	advertised	
Command	`beb5483e-36e1-4688-b7f5-ea07361b26a8`	write, write without response	phone to node
Response	`beb5483e-36e1-4688-b7f5-ea07361b26a9`	notify	node to phone
Status	`beb5483e-36e1-4688-b7f5-ea07361b26aa`	read	JSON, for example `{"status":1,"battery":-1,"build":3}`

Lines. Every message is one line, `TYPE:SUBTYPE:PAYLOAD\n`. Keep the second colon even when the payload is empty (`CMD:ping:`). Arguments are `key=value` pairs separated by commas.

- Send nothing on Response until the phone enables notifications.
- You may pack several lines into one notify, and a long line may span several notifies; the phone splits on `\n`. Keep a notify within MTU minus 3 (the phone asks for MTU 247, so up to 244 bytes).
- Keep each line you send under 160 bytes. Accept command writes up to 512 bytes (a long plan arrives as a long write).
- In names and SSIDs, replace commas, CR, LF and NUL with `_`, and cut them to 24 bytes.

Line	Meaning
`CMD:<name>:<args>`	phone to node
`RSP:<name>:OK`	command accepted; `RSP:<name>:ERROR[:reason]` if not

Line	Meaning
`STATUS:1:Ready` / `STATUS:2:Busy`	idle / scanning
`DATA:<kind>:<fields>`	a sighting or a heartbeat
`TIME:REQUEST:`	node to phone, every 5 s until the clock is set
`TIME:SYNC:2026-10-14T17:20:05Z`	phone to node, UTC ISO-8601

Connecting: hello, owner lock, clock

The phone connects, asks for MTU 247, enables notifications on Response, then writes these three lines:

1. `TIME:SYNC:<UTC time>`: set your clock from it (seconds are enough).
2. `CMD:ack:n=<seq>`: only if the phone already knows your board (see Sightings).
3. `CMD:hello:v=6`: the hello.

A board with no owner answers `RSP:hello:OK` and starts heartbeats straight away. The phone must see a `DATA:BEAT` line within 4 seconds of the hello, or it lets the board go.

Owner lock. The first heartbeat without the owned flag (64) makes the phone send `CMD:own:k=<32 hex chars>`. Store those 16 bytes in flash as the owner key, set flag 64 in your heartbeats and answer `RSP:own:OK`. From then on, the board belongs to that phone (and any phone the owner copies the key to).

When an owned board gets a plain hello, it answers with a challenge instead of OK:

```
RSP:hello:ERROR:owned,o=<owner id, 4 hex>,c=<nonce, 16 hex>
```

The owner's phone replies `CMD:hello:v=6,a=<tag, 16 hex>`. If the tag matches, answer `RSP:hello:OK`, then the phone resends the time and ack. Until the tag is proved, accept only `hello` and `ping`. Answer any other command with `RSP:<name>:ERROR:owned` (ignore acks and TIME:SYNC silently), send no sightings or heartbeats, and drop the link after 15 seconds. A wrong tag gets a fresh challenge.

```
owner id  = HMAC-SHA256(key, "swarm-owner") first 2 bytes, as a little-endian
u16
hello tag = HMAC-SHA256(key, "hello" + nonce[8] + address[6]) first 8 bytes
```

- `nonce` is 8 fresh random bytes per challenge.

- `address` is your board's Bluetooth address as the phone sees it, most significant byte first (`AA:BB:CC:DD:EE:FF` is AA, BB, … FF). Use the address you advertise with.
- All hex is lower case.

Freeing a board. Give users a way to forget the owner (the board becomes new again). Swarm boards do it on a 5-second BOOT button hold, the serial command `forget`, or `CMD:disown:` from the owner's proven link (answer `RSP:disown:OK`).

If you leave the owner lock out, never set flag 64 and answer `RSP:own:ERROR`. Your board then works with any Swarm phone that finds it.

Heartbeat

After a successful hello, send a heartbeat every 3 seconds and right after every command you accept (except acks). The phone compares it with what it wants and sends only the commands that differ, so the heartbeat is how your board reports its state.

```
DATA:BEAT:f=<flags>,h=<hash>,m=<run>,s=<session>,n=<node>,b=<backlog>,l=
<lost>,q=<nextSeq>,t=<oldest>,v=<build>
```

Field	Meaning
`f`	flags, added together: 1 scanning, 8 can scan 5 GHz, 16 queue spills to flash, 32 clock set since boot, 64 has an owner
`h`	hash of the plan you hold, decimal (see Reference)
`m`	run mask you're obeying: 1 Wi-Fi, 2 BLE, 3 both, 0 stopped
`s`	scan session number from the last `run`
`n`	your node number (0 = none yet)
`b`	sightings queued and not yet acknowledged
`l`	sightings dropped because the queue overflowed
`q`	the seq the next sighting will get
`t`	the oldest seq still in your queue (lets the phone skip a gap after a reboot)

Field	Meaning
`v`	your firmware build number (optional)
`bv`, `tc`, `up`, `hp`	optional health: battery mV (0 = unknown), chip temperature °C (-128 = unknown), minutes since boot, free RAM in KB

Keep flag values 128 and up clear; they are taken.

Example: `DATA:BEAT:f=105,h=8195,m=3,s=12,n=2,b=40,l=0,q=1873,t=1833,v=3`

Commands from the phone

One write can hold several lines; handle each in order. Answer every command except `ack` with an `RSP` line, then send a heartbeat. An unknown command gets `RSP:<name>:ERROR:unknown command`.

Command	What to do	Answer
`CMD:slot:n=<node>`	Store the node number in flash; advertise as `WDN<node>`	`RSP:slot:OK`
`CMD:plan:f=<flags>,ch=<ch>.<dwell>/<ch>.<dwell>/...`	Store the plan in flash exactly as sent. `f`: 1 Wi-Fi, 2 BLE. Each entry is a Wi-Fi channel and how long to listen there in 10 ms steps (`6.25` = channel 6 for 250 ms). Up to 48 entries	`RSP:plan:OK`
`CMD:run:m=<mask>,s=<session>`	Start or stop scanning. `m`: 1 Wi-Fi, 2 BLE, 0 stop. Keep it in flash, so a reboot or a lost phone doesn't stop the scan. Stamp new sightings with session `s`	`RSP:run:OK`, then `STATUS:2:Busy` or `STATUS:1:Ready`

Command	What to do	Answer
`CMD:ack:n=<seq>`	The phone has every sighting before `seq`: delete them	none, no heartbeat
`CMD:own:k=<hex32>`	See the owner lock	`RSP:own:OK` or `RSP:own:ERROR:owned`
`CMD:disown:`	Forget the owner	`RSP:disown:OK`
`CMD:ping:`	Nothing	`RSP:ping:OK`

The plan. The phone spreads the channels over all nodes, so following the plan is what keeps nodes from scanning the same channels. Hop through the listed channels in order, listening on each for its dwell time, and scan BLE when `f` has 2. A plan can include 5 GHz channels only if your heartbeat says you can (flag 8).

If your firmware can't follow a channel list, store the plan anyway so the hash in your heartbeat matches, and scan what you can. Otherwise the phone sends the plan again after every heartbeat.

Scanning keeps going without the phone. Once told to run, keep scanning and queueing until told to stop, even if the link drops or the board reboots. The phone collects the backlog when it's back in range.

Sightings and the queue

Every sighting is one line, with three Swarm fields on the end:

```
DATA:AP:<SSID>,<BSSID>,<channel>,<RSSI>,[<security>],q=<seq>,t=<unix>,s=
<session>
DATA:BT:<name>,<MAC>,<RSSI>,BLE,q=<seq>,t=<unix>,s=<session>
```

- `security` is one of `OPEN`, `WEP`, `WPA_PSK`, `WPA2_PSK`, `WPA_WPA2_PSK`, `WPA2_ENTERPRISE`, `WPA3_PSK`, `WPA2_WPA3_PSK`, `UNKNOWN`, in square brackets.
- MACs as `aa:bb:cc:dd:ee:ff`. An empty SSID or name stays empty.
- `t` is the unix second when you saw it, from the clock the phone set. The phone looks up its GPS track at that time, so a sighting with `t=0` can't be placed on the map. Hold sightings until the clock is set, or convert their time once it is.
- `s` is the session from the last `run`.

Example: `DATA:AP:HomeNet,a4:2b:b0:12:34:56,6,-67,[WPA2_PSK],q=1873,t=1792000805,s=12`

Sequence numbers. Give each queued sighting the next `seq`, counting up by one, and never reuse one, even across reboots (save a reserved block of seqs in flash and skip ahead after a reboot; the `t` field of the heartbeat tells the phone where your queue now starts).

The queue. Keep every sighting until the phone acknowledges it, in RAM and, if you can, spilling to flash. If the queue fills, drop the oldest and count them in the heartbeat's `l`.

Upload. After each hello, send from the oldest sighting the phone hasn't acknowledged, oldest first, as fast as the link allows. The phone answers `CMD:ack:n=<seq>` (at most every half second) meaning "I have everything before seq": delete those. If you've sent everything and the ack hasn't moved for about 10 seconds, start again from the oldest unacknowledged one. The phone drops repeats by seq, so sending one twice is harmless.

Duplicates. Don't queue the same thing over and over: Swarm boards queue a given access point again only after 5 minutes and a given BLE device after 30 seconds. The phone keeps one row per device per drive anyway.

Step by step

- [ ] Advertise the service UUID, connectable, named `WDU` / `WDN<node>`.
- [ ] Add the Command, Response and Status characteristics; split incoming writes on `\n`.
- [ ] Handle `TIME:SYNC` and send `TIME:REQUEST:` every 5 s until the clock is set.
- [ ] Answer `CMD:hello:v=6` with `RSP:hello:OK`, then start heartbeats (every 3 s and after each command).
- [ ] Handle `slot`, `plan`, `run`, `ack`, `ping`; keep node number, plan and run state in flash.
- [ ] Queue sightings with seq numbers; send them as `DATA:AP` / `DATA:BT` lines with `q`, `t`, `s`; delete on ack; resend from the oldest unacked after a hello or a stalled ack.
- [ ] Owner lock: `own`, the hello challenge, `disown`, a way to forget the owner.
- [ ] Check your plan hash and hello tag against the test vectors below.

Testing with the app. Power the board, open Swarm and tap **FIND MY NODES** on the BASIC tab. Within a few seconds it should show up on the SWARM tab as the next node number and say IN SYNC once its plan hash matches. Tap **START** on BASIC: the board

should get `CMD:run:m=3,...`, start scanning, and its sightings should appear in the live feed on ADVANCED, DRIVE. Walk the board out of range and back: its queued sightings should upload and the backlog count fall to zero.

If the board connects but never appears, the phone didn't get a heartbeat within 4 seconds of the hello. If it keeps showing SYNCING PLAN, your heartbeat's `h` doesn't match the plan you were sent.

Reference

Plan hash. 32-bit FNV-1a over single bytes (`flags & 3`, the entry count, then each channel and its dwell), folded to 16 bits. Dwell is the number from the plan line (10 ms steps, 2 to 255). Send it in decimal.

```
uint16_t plan_hash(uint8_t flags, const uint8_t *ch, const uint8_t *dwell,
uint8_t n) {
  uint32_t h = 2166136261u;
  #define MIX(b) h = (h ^ (uint8_t)(b)) * 16777619u
  MIX(flags & 3);
  MIX(n);
  for (uint8_t i = 0; i < n; i++) { MIX(ch[i]); MIX(dwell[i]); }
  #undef MIX
  return (uint16_t)((h >> 16) ^ (h & 0xFFFF));
}
```

Test vectors.

Input	Expected
Plan `CMD:plan:f=1,ch=1.25/6.25/11.25`	`h=8195`
Owner key `000102030405060708090a0b0c0d0e0f`: owner id	`576a`
Same key, nonce `0011223344556677`, address `AA:BB:CC:DD:EE:FF`: hello tag	`b2e51fdac2f3fa74`

Numbers to match.

Constant	Value
Protocol version in hello	6
Heartbeat interval	3 s
Time request interval (until set)	5 s
Phone waits for the first heartbeat	4 s after hello
Unproven link dropped after	15 s
Longest line sent / command accepted	160 / 512 bytes
Name or SSID kept	24 bytes
Plan entries	48
2.4 GHz channels in the auto plan	1 to 11
5 GHz channels in the auto plan	36 to 64, 100 to 144, 149 to 165 (every 4th)